

Proyecto de Paralelización con Open-MP: Ecuación de Poisson en 2D

Camilo Huertas Sebastián Rodríguez
Proyecto Curricular de Física
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia

24 de marzo de 2026

Objetivo

Aplicar diferentes directivas de OpenMP para paralelizar un código secuencial que resuelve la ecuación de Poisson 2D mediante diferencias finitas, y analizar el impacto de cada estrategia en el rendimiento y estructura del programa.

Especificaciones del Sistema de Pruebas

Todas las pruebas de rendimiento registradas en este informe fueron ejecutadas en un equipo con la siguiente configuración de hardware:

- **Procesador:** AMD Ryzen 7 7730U with Radeon Graphics.
- **Arquitectura:** x86_64.
- **Núcleos físicos:** 8.
- **Hilos lógicos (*Threads*):** 16.

Nota metodológica: Al no especificar un límite en el código fuente ni mediante la variable de entorno `OMP_NUM_THREADS`, OpenMP asignó por defecto el uso del total de **16 hilos lógicos** para todas las directivas de paralelización.

Preparación: Ejecución Secuencial

Al ejecutar el código base secuencial, se obtuvieron los siguientes resultados:

- **Tiempo de ejecución:** 12.3298 segundos.
- **Número de iteraciones:** 49367.
- **Número de threads:** 1.

Actividad 1: `#pragma omp parallel for`

Se paralelizó el bucle principal en `solve_poisson` aplicando `#pragma omp parallel for` y utilizando `reduction(max:delta)` para evitar condiciones de carrera.

- **Tiempo de ejecución:** 1.14783 segundos.

- **¿Hubo diferencia en el número de iteraciones?** Sí, bajó a 49360 iteraciones.
- **¿Por qué es necesaria la cláusula `reduction`?** Es fundamental porque múltiples hilos están calculando el error local y necesitan actualizar la variable compartida `delta`. Sin `reduction`, habría condiciones de carrera y pérdida de datos al sobrescribir el valor.
- **¿Hubo mejora de rendimiento?** Sí, la mejora fue drástica. Al utilizar los 16 hilos del procesador, se logró acelerar el proceso en un factor cercano a 10.7x respecto a la versión secuencial.

Actividad 2: `collapse(2)`

Se substituyó el bucle paralelo por `#pragma omp parallel for collapse(2) reduction(max:delta)`.

- **Tiempo de ejecución:** 1.82978 segundos.
- **¿Mejóro respecto a la versión anterior?** No, el tiempo aumentó en comparación a la Actividad 1.
- **¿Qué hace `collapse(2)`?** Fusiona los dos bucles anidados en un único espacio de iteración más grande antes de distribuirlo entre los hilos.
- **¿Es siempre recomendable su uso?** No. Como se observa en los resultados, si el bucle externo ya proporciona suficiente trabajo para mantener ocupados a los hilos, el `collapse` añade un sobre costo (overhead) de gestión de índices que termina ralentizando la ejecución.

Actividad 3: `sections`

Se paralelizaron `initialize_grid` y `poisson_source`.

- **Tiempo de ejecución:** 1.27695 segundos.
- **¿Vale la pena paralelizar funciones tan rápidas?** Generalmente no. El costo de crear la región paralela, sincronizar los hilos e iniciar las secciones supera los beneficios de ejecución paralela para funciones que computacionalmente son tan ligeras.

Actividad 4: `parallel` y `for` separados

Se separó la región paralela del bucle `for` y se evaluaron distintas estrategias de `schedule`.

- **Tiempo de ejecución:** 1.16695 segundos.

Actividad 5: `nowait`, `barrier`, `single`, `critical`

- **Tiempo con control de sincronización:** 1.19588 segundos.
- **¿`critical` ralentizó la ejecución?** Sí. Forzar secciones críticas obliga a que los hilos se escolten y pasen uno a la vez, eliminando el beneficio de concurrencia y agregando bloqueos.

Actividad 6: Conteo de iteraciones

Se evaluó el rendimiento utilizando `critical` y `atomic` para proteger el incremento de una variable compartida.

- **critical:** 1.18077 segundos.
- **atomic:** 1.41786 segundos.
- **Diferencias y Casos de Uso:** Curiosamente, `critical` fue ligeramente más rápido en este entorno. En general, `atomic` aprovecha instrucciones de hardware y es preferible para operaciones matemáticas simples (como un incremento ++), mientras que `critical` es necesario cuando el bloque a proteger contiene múltiples líneas de código o estructuras complejas.

Actividad 7: Exploración con tareas (task)

- **Tiempo de ejecución:** 23.2788 segundos.
- **Iteraciones:** 49367.
- **Conclusión:** El rendimiento usando `task` se degradó severamente (casi el doble de lento que la versión secuencial).
- **Ventajas y Desventajas:** La principal ventaja de `task` es su flexibilidad para estructuras irregulares (como recorrer árboles o listas enlazadas). La gran desventaja, evidente en este caso, es el enorme sobrecosto de crear y empaquetar una tarea por cada bloque. Para arreglos regulares y matrices bidimensionales densas, `parallel for` es inmensamente superior.

Actividad Final: Comparación de Estrategias

Versión	Directiva usada	Tiempo (s)	Iteraciones	Observaciones
Secuencial	N/A	12.33	49367	Ejecución base lenta. 1 hilo.
Paralelo básico	<code>parallel for</code>	1.15	49360	Mejor rendimiento global (16 hilos).
Colapsado	<code>collapse(2)</code>	1.83	49360	Overhead perjudica el tiempo.
Sections	<code>sections</code>	1.28	49360	Funciones muy pequeñas.
Schedule	<code>schedule</code>	1.17	49360	Muy similar al paralelo básico.
Sincronización	<code>nowait/critical</code>	1.20	49360	Leve penalización de tiempo.
Conteo (critical)	<code>critical</code>	1.18	49360	Ligeramente más rápido que <code>atomic</code> .
Conteo (atomic)	<code>atomic</code>	1.42	49360	Mayor overhead en este contexto.
Tareas	<code>task</code>	23.28	49367	Degradación severa del rendimiento.

Conclusiones Generales

1. **¿Cuál fue la versión más rápida?**

La Actividad 1, usando un `#pragma omp parallel for` básico con reducción, con 1.14783 segundos.

2. **¿Qué directiva es más fácil de usar?**

`#pragma omp parallel for`. Basta con colocarla sobre el bucle principal y manejar correctamente el `reduction` para tener un código robusto.

3. **¿Hubo directivas que empeoraron el rendimiento? ¿Por qué?**

Sí, la directiva `task` (23.28s) empeoró masivamente el tiempo, y `collapse(2)` (1.83s) fue peor que el bucle normal. Esto ocurre porque el sobre costo administrativo de los hilos de OpenMP al organizar tareas y aplanar bucles superó la ventaja computacional.